Proyecto MiBici: propuesta y base reproducible

Primera entrega del proyecto integrador de MLOps

Tecnológico de Monterrey

Curso: TC5061 · Operaciones de Machine Learning

Equipo 09

Mariana Judith Carrera Rojas · A01840021

José Manuel Barrera Mendoza · A01797854

Luis Fernando Barranco Cruz · A01797868

David Carrera Castillo · A01840136

Fecha: 18 de octubre de 2026

Repositorio: https://github.com/a01840021-creator/TC5061-Proyecto-Equipo9

Resumen

Este documento presenta la propuesta técnica y la base reproducible del proyecto MiBici del equipo 09. El proyecto utiliza seis meses de datos abiertos —abril a septiembre de 2026— para responder tres preguntas operativas: pronosticar la demanda de viajes por estación, identificar riesgo de vaciado o saturación y agrupar estaciones con patrones de uso semejantes. La solución integra una ingesta automatizada, datos procesados en Parquet, versionado mediante SHA-256, feature engineering sin fuga temporal, baselines reproducibles y seguimiento de experimentos con MLflow.

La ingesta obtuvo 2,294,886 registros crudos. Después de eliminar 758 duplicados exactos y 3,954 viajes asociados con estaciones ausentes de la nomenclatura, quedaron 2,290,174 viajes y 484 estaciones. La versión reproducible del conjunto de datos es 0c4b34e5425a.

1. Formulación del problema y ML Canvas

P1. Demanda por estación

Se predice el número de viajes que se iniciarán en cada estación durante cada hora del día siguiente. Es un problema de regresión con granularidad estación–hora y horizonte de 24 horas. El resultado sería utilizado por el operador para distribuir la flota y reforzar las franjas con mayor demanda.

P2. Riesgo de vaciado o saturación

Se clasifica si una estación estará en riesgo de quedarse sin bicicletas o sin anclajes en las siguientes dos horas. Debido a que MiBici no publica inventario en tiempo real, el riesgo se aproxima mediante el saldo neto futuro —llegadas menos salidas— y la capacidad de la estación. La etiqueta toma valor 1 cuando el valor absoluto del saldo de las siguientes dos horas alcanza al menos 30 % de la capacidad. El equipo de rebalanceo usaría el resultado para priorizar estaciones y decidir si debe llevar o retirar bicicletas.

P3. Agrupamiento de estaciones

Se agrupan estaciones según su distribución horaria de salidas, proporción de uso en fin de semana y balance relativo entre salidas y llegadas. El resultado permite identificar perfiles

funcionales y ubicarlos geográficamente para apoyar decisiones de expansión y planeación urbana.

ML Canvas resumido

- Propuesta de valor: anticipar demanda y desbalances para mejorar la disponibilidad del servicio y comprender el comportamiento territorial del sistema.
- Decisiones: asignación de flota, rutas de rebalanceo y priorización de expansión.
- Generación de predicciones: P1 una vez al día; P2 cada hora; P3 al cierre de cada periodo de análisis.
- Monitoreo: error de demanda, recuperación de eventos de riesgo, estabilidad de clusters y cambios en volumen o estaciones.

El ML Canvas completo, con sus diez bloques, está documentado en `references/ml_canvas.md` del repositorio. Esta síntesis debe leerse junto con ese archivo como evidencia integral de la formulación del proyecto.

2. Datos, gobierno y partición temporal

Fuente: portal de datos abiertos de MiBici. Se utilizaron los archivos mensuales de abril a septiembre de 2026 y la nomenclatura de estaciones de septiembre de 2026. Las URL exactas se centralizan en mibici/config.py.

Justificación del periodo: se eligieron los seis meses consecutivos de abril a septiembre de 2026 porque constituyen un periodo reciente con volumen suficiente para capturar patrones horarios y semanales, y permiten reservar septiembre como una prueba futura sin romper el orden temporal.

Privacidad: usuario_id, género y año de nacimiento se eliminan durante la ingesta porque no son necesarios para las preguntas del proyecto. Los datos originales, las bases de MLflow, los modelos serializados y las credenciales están excluidos de Git.

Condiciones de uso: el portal de MiBici no publica una licencia abierta específica y señala “Todos los derechos reservados”. En consecuencia, el equipo utiliza los datos exclusivamente con fines académicos, no redistribuye los archivos originales y excluye datos personales del procesamiento y del repositorio.

Calidad: además de duplicados y estaciones no reconocidas, features.py excluye 81,947 registros de menos de un minuto que empiezan y terminan en la misma estación, interpretados como préstamos fallidos o devoluciones inmediatas.

Partición: abril–agosto de 2026 se utiliza para entrenamiento y septiembre para prueba. La división es temporal, nunca aleatoria, con el propósito de impedir que los modelos utilicen información futura.

3. Pipeline de Machine Learning

Fuentes MiBici → descarga reproducible → normalización y validación → Parquet + manifiesto SHA-256 → feature engineering por pregunta → partición temporal → baselines y candidatos → métricas y artefactos → MLflow Tracking.

Ingesta: mibici.dataset descarga automáticamente los archivos, normaliza columnas y codificación, convierte fechas, elimina duplicados, filtra estaciones no reconocidas, retira datos personales y produce viajes.parquet, estaciones.parquet y el manifiesto versionado.

Feature engineering: P1 completa el producto cartesiano estación–hora e incorpora rezagos de 24, 48 y 168 horas. P2 calcula salidas, llegadas, saldo neto, rezagos y una ventana futura de dos horas. P3 construye una distribución de uso de 24 horas, uso en fin de semana y razón de salidas.

4. Baselines, modelos candidatos y métricas

P1 — Regresión

Baseline: demanda de la misma estación y hora observada una semana antes (rezago de 168 horas). Modelo candidato: Ridge con variables temporales y rezagos. Métricas: MAE y RMSE para cuantificar el error en viajes, WAPE para expresar error relativo, R^2 como referencia explicativa y sesgo para detectar sobreestimación o subestimación sistemática.

P2 — Clasificación

Baseline: regla de persistencia basada en el saldo de la hora previa respecto de la capacidad. Modelo candidato: regresión logística con balanceo de clases. Métricas: recall como prioridad operativa, precision para evitar movilizaciones innecesarias, F1, balanced accuracy y average precision por el desbalance de la clase positiva.

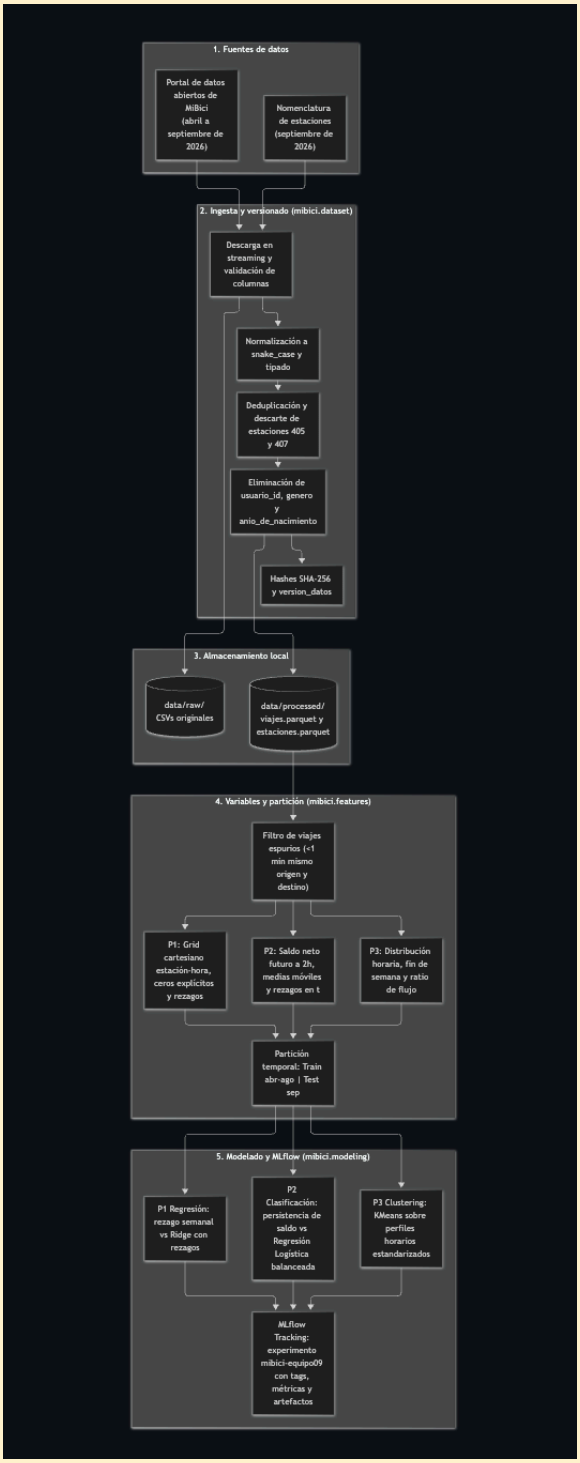
P3 — Clustering

Baseline: K-Means sobre perfiles estandarizados. **Modelo candidato:** agrupamiento jerárquico aglomerativo con enlace de Ward, comparado con K-Means; el valor de k se seleccionará mediante el coeficiente Silhouette. **Métricas:** Silhouette —mayor es mejor—, Davies–Bouldin —menor es mejor—, Calinski–Harabasz —mayor es mejor— e inercia —menor es mejor, aunque siempre disminuye al incrementar k—.

5. Arquitectura del sistema

La arquitectura separa fuentes, ingesta, almacenamiento, transformación, modelado y seguimiento. Esta modularidad reduce deuda técnica, permite reconstruir los datos y hace que cada corrida pueda vincularse con una versión de datos y un commit de Git.

Herramientas y justificación: la ingesta se implementa con Python y pandas por su capacidad para normalizar archivos CSV heterogéneos, mientras que PyArrow y Parquet proporcionan almacenamiento columnar eficiente para millones de viajes. hashlib con SHA-256 permite verificar la integridad y versión de cada archivo; scikit-learn ofrece interfaces consistentes para preprocesamiento, baselines, modelos y métricas; y MLflow Tracking conserva parámetros, resultados, etiquetas y artefactos de cada corrida. Cookiecutter Data Science separa datos, código, modelos y reportes, mientras que Git y GitHub aportan versionado, revisión mediante Pull Requests y trazabilidad del código. Estas decisiones siguen la reproducibilidad del ciclo MLOps descrita por Treveil et al. (2020) y reducen la deuda técnica asociada con dependencias y procesos ocultos señalada por Sculley et al. (2015). El detalle se conserva en `references/arquitectura.md`.



6. Experimento base en MLflow

Todos los runs pertenecen al experimento mibici-equipo09 y registran el periodo de análisis, versión de datos, semilla, modelo, tamaños de entrenamiento y prueba, pregunta, equipo, autor y commit de Git. El código se ejecutó desde el repositorio mediante python -m mibici.modeling.train.

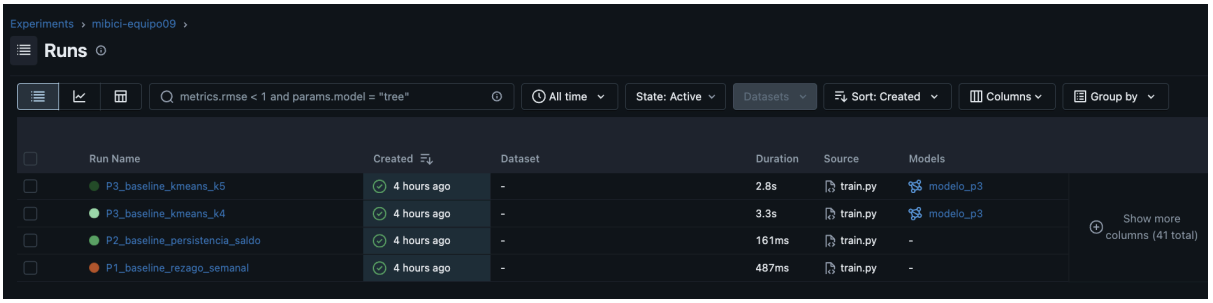
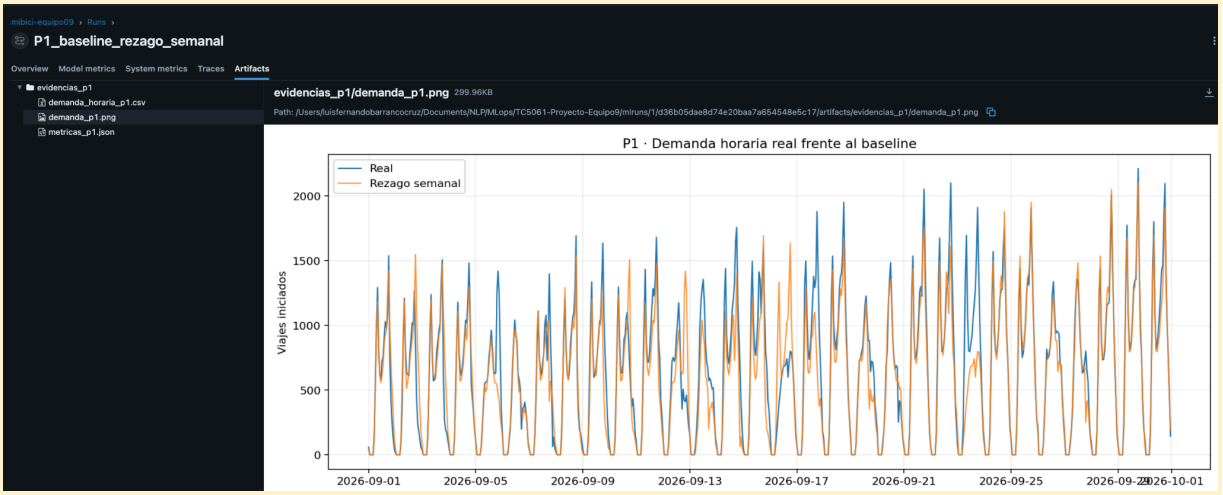


Figura 1. Lista de runs registrados en el experimento mibici-equipo09.

6.1 Resultados P1





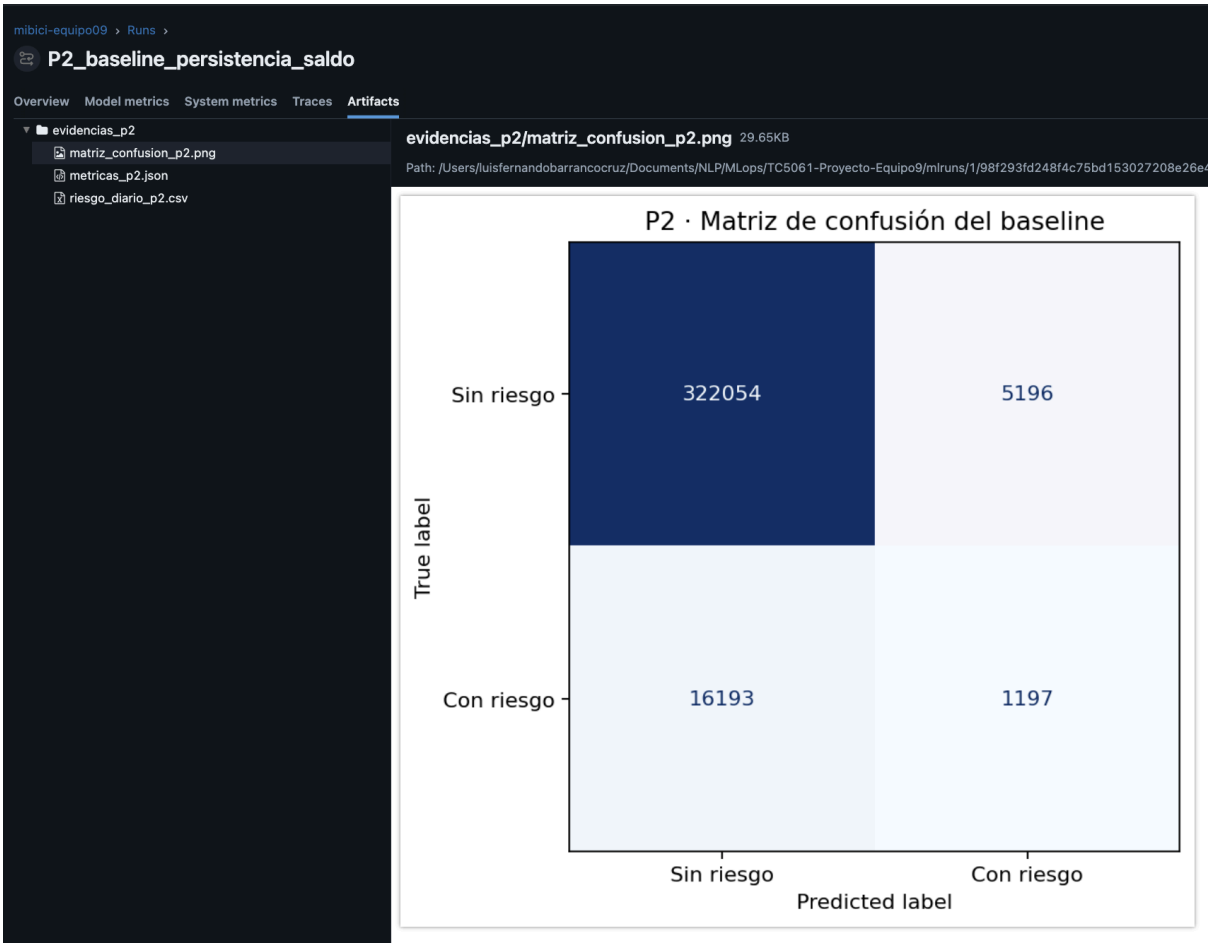
Resultados cuantitativos: el baseline obtuvo un MAE de 0.95 viajes por estación-hora y un R^2 de 0.44. Esto significa que el error absoluto promedio es inferior a un viaje por

observación y que el modelo explica 44 % de la variabilidad, por lo que constituye una referencia útil, aunque todavía deja margen de mejora.

El baseline utiliza como predicción la demanda observada en la misma estación y hora de la semana anterior. La gráfica permite identificar en qué periodos el patrón semanal representa adecuadamente la demanda y en cuáles aparecen desviaciones asociadas con cambios operativos, estacionales o de comportamiento.

6.2 Resultados P2





Resultado cuantitativo: el baseline alcanzó un recall de 0.07 para la clase de riesgo; es decir, solo identificó aproximadamente 7 % de los eventos positivos y omitió cerca de 93 %. El desempeño confirma que la regla de persistencia es una referencia débil y que se requiere un modelo candidato con mayor sensibilidad.

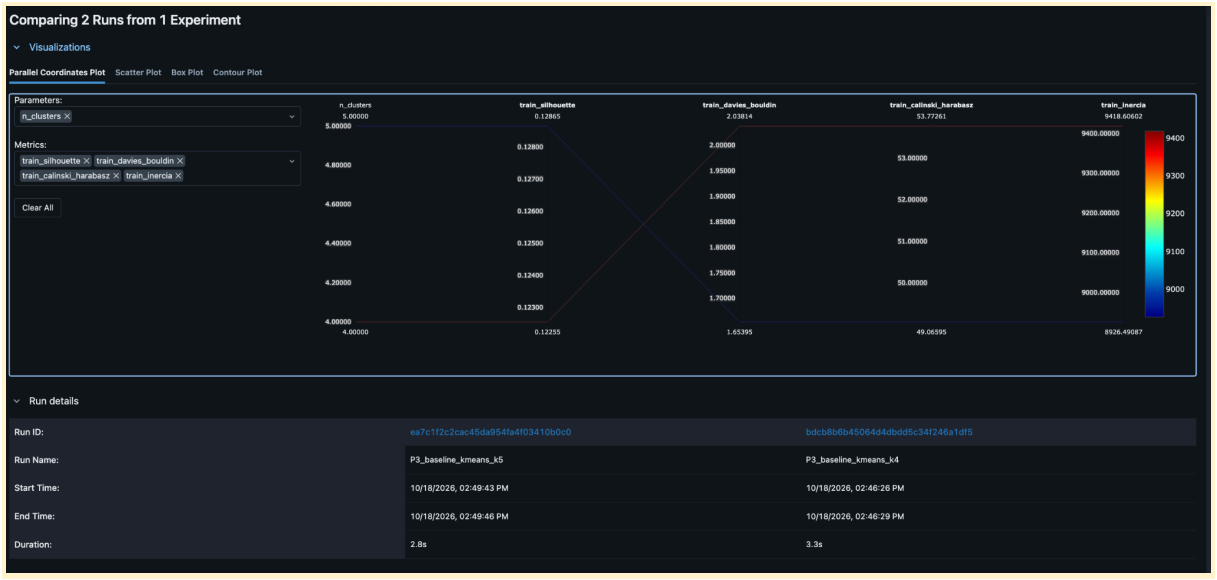
La matriz de confusión permite distinguir los eventos de riesgo detectados correctamente, los riesgos omitidos y las falsas alertas. Para esta aplicación se prioriza el recall de la clase positiva, porque un falso negativo representa una estación en riesgo que no sería atendida por el equipo de rebalanceo.

6.3 Resultados P3 y comparación de configuraciones

Se compararon K-Means con k=4 y k=5. Para k=5 se obtuvo Silhouette de 0.12865, Davies–Bouldin de 1.65395 e inercia de 8,926.49. Para k=4 se obtuvo Silhouette de 0.12255, Davies–Bouldin de 2.03814 e inercia de 9,418.61. Calinski–Harabasz favoreció ligeramente a k=4 —53.77 frente a 49.07—.

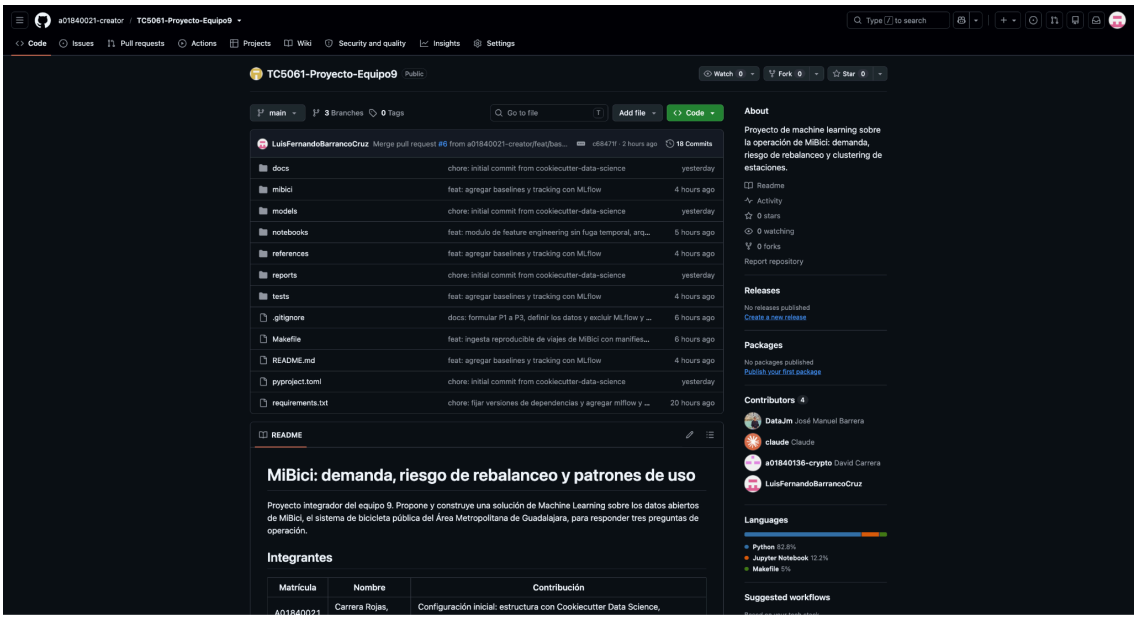
En conjunto, k=5 es la alternativa preferida porque presenta mayor cohesión y separación según Silhouette, menor similitud entre clusters según Davies–Bouldin y menor dispersión

interna. Sin embargo, los valores bajos de Silhouette muestran que los patrones no forman grupos fuertemente separados; los clusters deben interpretarse como segmentos operativos útiles y no como categorías naturales definitivas.



7. Reproducibilidad y colaboración

El repositorio sigue Cookiecutter Data Science y separa datos, notebooks, código, modelos, referencias y reportes. Las dependencias están fijadas, las rutas y semillas se centralizan en config.py y la base SQLite de MLflow queda fuera de Git. El flujo de colaboración utiliza GitHub Flow, ramas por tarea, Conventional Commits y Pull Requests con revisión.



Comandos principales

```
python -m mibici.dataset
python -m mibici.modeling.train --pregunta all
python -m mibici.modeling.train --pregunta P3 --n-clusters 5
mlflow ui --backend-store-uri sqlite:///mlflow.db --host 127.0.0.1 --port 5000
```

8. Conclusiones

La primera entrega demuestra la viabilidad técnica del proyecto sobre más de 2.2 millones de viajes. La ingesta reproducible, el manifiesto de datos, la separación temporal y el tracking en MLflow permiten rastrear qué datos, configuración y código produjeron cada resultado. Los baselines establecen referencias transparentes para evaluar mejoras posteriores sin confundir complejidad con valor.

El proyecto también explicita sus principales limitaciones: no existe inventario histórico por estación, P2 utiliza una aproximación basada en flujo neto y los clusters presentan separación moderada. Estas limitaciones no invalidan el análisis, pero delimitan el alcance operativo de sus conclusiones y orientan la siguiente iteración hacia modelos candidatos, evaluación temporal continua y validación con conocimiento del operador.

9. Referencias

Dorard, L. (2018, 9 de marzo). From data to AI with the Machine Learning Canvas (Part I). Own Machine Learning. https://medium.com/louis-dorard/from-data-to-ai-with-the-machine-learning-canvas-part-i-d171b867b047

GitHub. (s. f.). GitHub Flow. https://docs.github.com/en/get-started/using-github/github-flow

MiBici. (s. f.). Datos abiertos [Conjunto de datos]. Sistema de Bicicleta Pública del Área Metropolitana de Guadalajara. https://mibici.net/es/datos-abiertos/

MLflow. (s. f.). MLflow Tracking. https://mlflow.org/docs/latest/ml/tracking/

Scikit-learn. (s. f.). Metrics and scoring: Quantifying the quality of predictions. https://scikit-learn.org/stable/modules/model_evaluation.html

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden technical debt in machine learning systems. Advances in Neural Information Processing Systems, 28.

Treveil, M., Omont, N., Stenac, C., Lefevre, K., Phan, D., Zentici, J., Lavoillotte, A., Miyazaki, M., & Heidmann, L. (2020). Introducing MLOps: How to scale machine learning in the enterprise. O'Reilly Media.

10. Declaración de uso de inteligencia artificial

El equipo utilizó herramientas de inteligencia artificial generativa como apoyo durante el desarrollo del proyecto: ChatGPT y Codex, de OpenAI, y Claude, de Anthropic. Estas herramientas se emplearon para apoyar la estructuración del proyecto, la refactorización y depuración del código, la interpretación de errores, la revisión documental y la formulación de sugerencias técnicas. Claude aparece como coautor en algunos commits para transparentar su participación asistida en cambios de código y documentación. Las personas integrantes del equipo ejecutaron y probaron el código, verificaron las corridas de MLflow, validaron las métricas y los artefactos, revisaron las propuestas generadas y asumieron la responsabilidad final por las decisiones, el contenido y la entrega.